Type Expansion in SKJS: the Instance Layer, the Expansion Cache, and Axiom **L-EXP**

Deep Dive C of the SKJS Soundness Series

Models revision `cf5a6bc7` of `main` (2026-07-09).
Companion to the main soundness paper.

Abstract

The main soundness paper assumes axiom **L-EXP**: expansion of a type to its instance-plus-clauses view preserves or enlarges its semantics, with residual instances keeping theirs. This deep dive models expansion precisely as a worklist over the instance layer, proves the containment $[\![t]\!]_\rho \subseteq [\![\mathsf{expand}(t)]\!]_{\rho'}$ for the expansion modes the refinement machinery actually uses, and — as its centerpiece — turns the `seenExpansion` cache-soundness comment (`TTypeSubst.sk:17--31`) into a precise invariance theorem. The audit yields a result *stronger* than the comment claims for type aliases: normalization is *binding-blind* — every substitution it consults for variable bindings is freshly created empty — so a cached alias expansion is a function of the instance alone and is immune to the freeze-window staleness that threatens the other memo caches. It also yields corrections: the comment's "`inter` never resolves an unbound `TVar`" is false for variables nested in compound operands (harmless where it matters, for a stated reason); the class-instantiation path is *not* binding-blind and remains an open obligation; and contravariant-mode expansion does not satisfy **L-EXP** at all, forcing a scope edit to the axiom. Section 7 gives the resulting ledger edits for the main paper's Assumptions table.

Contents

1 Purpose **2**

2 Minimal Recap **2**
- 2.1 Representation and semantics . . . 2
- 2.2 The substitution state, condensed . . . 3
- 2.3 The axiom under study . . . 3

3 Expansion as a Function **4**
- 3.1 The worklist . . . 4
- 3.2 The plumbing is sound . . . 5

4 Per-Construct Containment **5**
- 4.1 Type variables . . . 5
- 4.2 Named instantiations (`TSInst`) . . . 6
- 4.3 Unexpanded intersections (`TInter`) . . . 7
- 4.4 Delayed array literals (`TArrayLit`) . . . 7
- 4.5 The containment theorem . . . 8

5 Centerpiece: Soundness of the `seenExpansion` Cache **8**
5.1 Symbolic arguments, precisely . . . 8
5.2 The alias path is binding-blind . . . 8
5.3 Verifying the comment's three claims . . . 10
5.4 Where the proof stops: the class path . . . 11

6 Alias Cycles and Termination **12**

7 Status and Ledger Edits **12**
7.1 Decomposition of L-EXP . . . 12
7.2 Registry . . . 13
7.3 Findings from failed proof attempts (index) . . . 14

1 Purpose

Row addressed. **L-EXP** (the main paper's Assumptions and Proof Obligations table records the mode-scoped PROVED status this paper establishes): "Expansion preserves or enlarges semantics; residual instances keep theirs," anchored at `TType::expand`. Every refinement operator of the main paper runs on the expanded view (its Definition "Memberwise Refinement"), and the proof of its memberwise-refinement lemma consumes **L-EXP** in its first step; the axiom is therefore load-bearing for every branch-soundness result.

What this paper does. (i) models expand as a function of a type, a substitution state, and a variance mode, exactly following `TType::expandToVectors`; (ii) proves the containment per instance construct, splitting off two named interface assumptions where the argument crosses into `TEnv` (normalization) and array-literal commitment; (iii) proves the `seenExpansion` cache-invariance theorem for the type-alias path and names the class path as an obligation; (iv) records the state evolutions expansion can cause and defers their transport to the main paper's **L-MONO**; (v) states precise ledger edits (Section 7). Failed proof attempts are reported as numbered Findings.

Conventions. Notation, semantics, and the status vocabulary (PROVED / ASSUMED / OPEN / VIOLATED) are the main paper's. All code references are against the pinned revision `cf5a6bc7`; line numbers are cited where they disambiguate. "Paper B" is the deep dive on union and intersection (**L-UNION**/**L-INTER**), which this paper cites and does not re-prove.

2 Minimal Recap

2.1 Representation and semantics

A TType is a pair (I, C) of an instance set $I \in$ TInsts and clauses $C \in$ Clauses, with $[\![(I,C)]\!]_\rho = [\![I]\!]_\rho \cup [\![C]\!]_\rho$ (main paper, Definition "Clause, Clauses, TType"). A clause $\langle V \mid S \rangle$ pairs type parameters with an SType union or $\top$; the top clause $\langle \emptyset \mid \top \rangle$, if present, is alone. The instance layer TInst has four constructors: type variables α (`TVar`, carrying a finite set of *occurrence-level* constraint types — `TVar::setConstraint` is a functional update returning a new `TVar` value with the same identifier — registry row SETCONSTRAINT-FUN, deep dive E), unexpanded named instantiations `TSInst`$(n, \bar{a})$, unexpanded intersections `TInter`$(\{t_1, \ldots, t_k\})$, and delayed array literals `TArrayLit`$(\alpha, \mathit{elts}, \mathit{fresh})$.

impl: `TType.sk / TType, TInsts, Clause, Clauses`

impl: `TType.sk / TVar (2576), TSInst (2626), TArrayLit (2618); TTypeInter.sk / TInter (17)`

The main paper gives the instance layer an abstract, state-indexed semantics. We spell out the part this paper consumes:

Definition 1 (Instance Semantics Interface)**.** *For a substitution state ρ (Definition 2):*

(V1) $[\![\alpha^K]\!]_\rho \subseteq \bigcap_{c\in K}[\![c]\!]_\rho$ *for a variable occurrence* α^K *with constraint set* K*;*

(V2) if ρ *binds* α *(as* $\mathsf{SVOpen}(t)$ *or* $\mathsf{SVClosed}(t)$*), then* $[\![\alpha^K]\!]_\rho \subseteq [\![t]\!]_\rho$*;*

(I1) $[\![\mathsf{TInter}(\{t_i\})]\!]_\rho = \bigcap_i [\![t_i]\!]_\rho$ *(an unexpanded intersection* means *the intersection);*

(N1) $[\![\mathsf{TSInst}(n, \bar{a})]\!]_\rho$ *is the denotation of the named definition* n *at arguments* $\bar{a}$ *— related to expansion only through assumption* ***E-NORM*** *below;*

(A1) $[\![\mathsf{TArrayLit}(\alpha, elts, _)]\!]_\rho$ *is the set of runtime values of the originating array-literal expression — related to expansion only through assumption* ***E-ARR*** *below.*

(V2) is the main paper's $[\![\alpha]\!]_\rho \subseteq [\![\rho(\alpha)]\!] \cap \bigcap_c [\![c]\!]$ *(stated there for bound* α*); (V1) extends the constraint containment to* unbound *occurrences — that is the sharpened variable denotation of registry row TVAR-SEM (*OPEN*, owner B), which this paper therefore imports: the covariant-variable cases below are conditional on it.*

2.2 The substitution state, condensed

Definition 2 (State; get transitions used by expansion)**.** *ρ maps `TVarIDs` to* $\mathsf{SVOpen}(t)$ *or* $\mathsf{SVClosed}(t)$ *and carries the flag* freeze *(`freezeCandidates`,* not *part of the saved state). Expansion reads variables through* $\mathsf{get}(\alpha, var)$*: covariant get on an* unbound α *seals it to top ($\alpha \mapsto \mathsf{SVClosed}(\top)$, returning $\top$); on* $\mathsf{SVOpen}(t)$ *it seals to* $\mathsf{SVClosed}(t)$ *and returns t — except under* freeze *for a candidate, where it returns t* without *sealing; on* $\mathsf{SVClosed}(t)$ *it returns t. Contravariant get returns $\bot$ on unbound (no write) and the bound value otherwise. Both then intersect the result with the occurrence's constraint types via `TType::inter`. The full transition table is the main paper's Definition "Transitions".*

impl: `TTypeSubst.sk / Subst::get (188), freezeCandidates (89)`

The mutable `Subst` also carries four memo caches — `seenSub`, `seenInter`, `seenUnion`, and this paper's subject `seenExpansion` : $\mathsf{TSInst} \to \mathsf{TType}$ — all four part of the saved state: `save`/`restore` and `clone` copy bindings and caches *together*.

impl: `TTypeSubst.sk / SubstState (13--49), SavedSubst (51), clone (137)`

2.3 The axiom under study

Axiom 3 (L-EXP, as stated in the main paper)**.** *Expansion rewrites a type to an instance-plus-clauses view using the current substitution state, possibly evolving it ($\rho \to \rho'$, e.g. sealing a fresh covariantly-read variable to top). Then* $[\![t]\!]_\rho \subseteq [\![\mathsf{expand}(t)]\!]_{\rho'}$*, and residual instances keep their semantics.*

Two things the main paper leaves open, which this paper fixes: expansion takes a *mode* argument (`expandTVars : ?Variance`), and the containment holds for modes none and cov but *not* for contra (Section 4.1); and "residual instances" means exactly: unresolved type variables in mode none, plus unexpanded recursion cut-offs re-emitted by `instantiate` and `TInter` cycle guards (Sections 4.2, 4.3, 6).

3 Expansion as a Function

3.1 The worklist

Definition 4 (Expansion)**.** *For a mode* $m \in \{\mathsf{none}, \mathsf{cov}, \mathsf{contra}\}$ *(the implementation's* `expandTVars` $\in \{$`None()`, `Some(Covariant())`, `Some(Contravariant())`$\}$*), define* $\mathsf{expand}_m(t, \rho) = (I', C', \rho')$ *by the following worklist, mirroring* `TType::expandToVectors` *line by line. If* t *is top, the result is* $\top$ *(returned as* `None`*). Otherwise initialize* $C' := t.\mathsf{clauses}$ *(as a vector),* $I' := \emptyset$*, todo* $:= t.\mathsf{insts}$*, seen* $:= t.\mathsf{insts}$*. While todo is nonempty, pop* i *and compute a one-step expansion* $E_m(i)$*:*

- $i = \mathsf{TSInst}(n, \bar{a})$*: if cached,* $E_m(i) := \rho.$`seenExpansion`(i)*; else* $E_m(i) := \mathsf{instantiate}(n, \bar{a})$ *and the cache is extended (*$\rho.$`seenExpansion`$[i] := E_m(i)$*). Mode-independent.*
- $i = \mathsf{TInter}(\{t_j\})$*:* $E_m(i) := \mathsf{interN}(\{t_j\})$ *with* `tenv.expandInter` *set (Section 4.3). Mode-independent.*
- $i = \alpha^K$*: in mode* none*, push* α^K *onto* I' (residual) *and continue; in modes* cov/contra*,* $E_m(i) := \mathsf{get}(\alpha^K, m)$ *— the binding read intersected with the occurrence constraints, with the seal side effects of Definition 2.*
- $i = \mathsf{TArrayLit}(\alpha, elts, _)$*:* $E_m(i)$ *is the committed type* $\mathsf{peek}(\alpha)$ *if* α *is bound (no state change), else the mutable tuple type over elts (Section 4.4). Mode-independent.*

If $E_m(i)$ *is top, the whole expansion returns* $\top$*. Otherwise extend* C' *with* $E_m(i).\mathsf{clauses}$ *and push each instance of* $E_m(i).\mathsf{insts}$ *not in seen onto todo (adding it to seen). On termination,* $\mathsf{expand}_m(t, \rho) = (I', C', \rho')$ *where* ρ' *is the evolved state.* `TType::expand` *rebuilds the pair from* $\mathsf{TInsts::unionN}(I')$ *and* $\mathsf{Clauses::unionN}(C')$*;* `expandToClauses` *does the same in mode* `Some(variance)` *and returns the clause part.*

impl: `TType.sk / expandToVectors (634), expand (680), expandToClauses (692)`

Lemma 5 (State Evolutions of Expansion)**.** *The only substitution evolutions* expand_m *performs are: (i)* get *transitions — seal-to-top of unbound variables and value-preserving seals* $\mathsf{SVOpen}(t) \to \mathsf{SVClosed}(t)$ *(modes* cov*; mode* contra *performs no binding writes); (ii) memo-cache extensions (*`seenExpansion` *here;* `seenInter` */* `seenUnion` *inside the invoked operators); (iii) version bumps. In particular expansion never executes a* grow*: no binding value is ever unioned or replaced.*

Proof. By inspection of Definition 4 against the code: the `TVar` arm calls `Subst::get` only; the `TSInst` arm calls `tenv.instantiate(subst, subst, ...)`, and `instantiate` immediately repackages the state into a fresh `Subst::mcreate(tcenvb, baseSubst)` *copy* — its binding writes (constraint-check clones, class-body expansion) land in the copy and are dropped; only the caller's `seenExpansion` write at `TType.sk:653` touches the live state. The `TInter` and `TArrayLit` arms call `interN` (get-transitions inside `inter_`'s covariant expansions, plus `seenInter`) and `peek` (read-only). □

impl: `TEnv.sk / instantiate (110, 135); TType.sk / expandToVectors (631); TTypeSubst.sk / peek (224)`

Transport of already-installed types across these evolutions is exactly the main paper's axiom **L-MONO** (whose anchor names the `Subst::get` seal-to-top); we cite it rather than re-prove it. Note that the value-preserving seal $\mathsf{SVOpen}(t) \to \mathsf{SVClosed}(t)$ does not change any denotation under Definition 1 — (V2) reads the bound value irrespective of open/closed — so among expansion's evolutions only seal-to-top has semantic content, and it only *adds* a binding of value $\top$.

3.2 The plumbing is sound

Lemma 6 (Worklist Containment)**.** *Fix a mode m and suppose the one-step containment*

$$[\![i]\!]_{\hat{\rho}} \subseteq [\![E_m(i)]\!]_{\hat{\rho}'} \qquad (*)$$

holds for every instance i popped during the run, where $\hat{\rho} \to \hat{\rho}'$ is that step's evolution. Then $[\![t]\!]_\rho \subseteq [\![(I', C')]\!]_{\rho'}$, and residual instances appear in I' verbatim.

Proof. Let $v \in [\![t]\!]_\rho = [\![t.\mathsf{insts}]\!]_\rho \cup [\![t.\mathsf{clauses}]\!]_\rho$. All later evolutions are those of Lemma 5, so membership transports forward by **L-MONO**; we may therefore evaluate all containments at the final ρ'. If v is justified by the clause part, it is in the initial C', and Clauses::unionN only merges clauses keyed by their var-sets using STypes/clause union, which over-approximates set union by **L-UNION** (paper B); the constructor invariant that a top clause absorbs the rest, and `TType::create`'s dropping of I' when C' is top, only enlarge the result. If v is justified by an instance i, argue by induction on the number of worklist steps after i enters *seen*: when i is popped, $(*)$ moves v into $[\![E_m(i)]\!]$, i.e. into its clauses (already covered) or one of its instances i''; if i'' is fresh it enters the worklist and induction applies; if $i'' \in$ *seen*, then i'' was either already processed (induction applies to its earlier occurrence — one-step expansion of the same instance under the evolved state yields the same $E_m(i'')$, by cache determinism for `TSInst` (Theorem 18) and by value determinism of sealed reads for the other arms) or is currently queued. The residual `TVar` arm pushes i itself onto I'; the top short-circuit returns $\top$ with $[\![\top]\!] = \mathsf{Val} \supseteq$ everything. □

One genuine subtlety hides in the "already processed" case: a *recursive* named type re-emits its own `TSInst`, which the *seen* check then drops — the recursive occurrence contributes nothing new only under a least-fixpoint reading of recursive definitions. Section 6 isolates this.

Lemma 7 (mapSTypes Decomposition)**.** *`TType::mapSTypes`(m, f) equals: expand in mode m; apply f to each clause's `?STypes` (with a $\top$ expansion presented to f as the single top clause `None`); drop clauses whose image is empty; rebuild with* Clauses::unionN *and the untouched residual I'. Consequently the main paper's refinement primitive* $\mathsf{refine}^{g,d}$ *factors exactly as its Definition "Memberwise Refinement" describes, with this paper supplying the expansion step and paper B the rebuild unions; and `mapSTypes` with the identity mapper is `expand`.*

impl: `TType.sk / mapSTypes (388--414); TypeCheckRefine.sk / setLocalFilterMap (600, 615)`

Proof. By inspection: `mapSTypes` calls `expandToVectors`, maps f through `Clause::create` (which returns `None` exactly on an empty STypes image, implementing the drop), and rebuilds. The top case at line 396–399 presents `None` to f and produces either the replacement clauses or $\top$. `rawMap` is the substitution-free analogue used by generic transformers and performs no expansion at all (its `fInsts` may convert instances to clauses but has no access to a substitution); no refinement client reaches it, so it carries no **L-EXP** obligation.

impl: `TType.sk / rawMap (362)` □

4 Per-Construct Containment

4.1 Type variables

Lemma 8 (TVar One-Step Containment, modes none and cov)**.** *For every variable occurrence α^K and state ρ:*

(a) Mode none*: the step is the identity (α^K residual); containment and residual-semantics preservation are trivial.*

(b) Mode cov*, ρ binds α (open or closed, freeze or not): the step returns $t \sqcap K$ built by* `TType::inter` *from the bound value t and the occurrence constraints. By (V2), (V1) and* ***L-INTER*** *(paper B), $[\![\alpha^K]\!]_\rho \subseteq [\![t]\!]_\rho \cap \bigcap_K [\![c]\!]_\rho \subseteq [\![t \sqcap K]\!]_{\rho'}$, the last step reading the result at the post-step state via* ***L-MONO****. The seal $\mathsf{SVOpen}(t) \to \mathsf{SVClosed}(t)$, when it fires, is value-preserving.*

(c) Mode cov*, α unbound: the step seals $\alpha \mapsto \mathsf{SVClosed}(\top)$ and returns $\top \sqcap K$. By (V1) and* ***L-INTER****, $[\![\alpha^K]\!]_\rho \subseteq \bigcap_K [\![c]\!]_\rho \subseteq [\![\top \sqcap K]\!]_{\rho'}$; the new binding of value $\top$ makes (V2) hold vacuously at ρ'.*

Finding 9 (Contravariant expansion does not satisfy L-EXP)**.** *We first attempted to prove the one-step containment for all three modes. It fails in mode* contra *on an* unbound *variable:* $\mathsf{get}(\alpha, \mathsf{contra})$ *returns $\bot$ (no write), and $[\![\alpha^K]\!]_\rho \subseteq [\![\bot]\!] = \emptyset$ is false whenever the variable's envelope is inhabited. This is not a bug: a contravariant read is a deliberate* under*-approximation — "the lower bound gathered so far" — consumed by the subtyping/growth machinery (*`growTVars`*,* `maybeGetRestElementType`*), where the required direction is reversed; that is the main paper's Obligation "Validation Soundness", not this paper's. For a* bound *variable, case (b)'s argument goes through verbatim in mode* contra*, since* $\mathsf{get}(\mathsf{contra})$ *then returns the same bound value. Consequence: axiom* ***L-EXP*** *must be scoped to modes* none/cov *(the mode scoping of Axiom* ***L-EXP****; Section 7).*

impl: `TTypeSubst.sk / Subst::get (210--216); TTypeSubtyping.sk / growTVars`

Lemma 10 (Refinement Clients Use Only Modes none/cov)**.** *Every expansion performed by the refinement and constant-removal machinery of the main paper uses mode* cov *or mode* none*:*

- `setLocalFilterMap` *maps with* `Some(Covariant()) (TypeCheckRefine.sk:615);`
- *the truthiness residues go through* `residuePart` *with* `Some(Covariant()) (TType.sk:350);`
- `conditionVerdict` *and* `getSTypes`*-based readers use* `expandToClauses` *with* `Covariant()` *(`TType.sk:279, 631`);*
- `removeConstant` *expands with* `None()` *(`TType.sk:212`), leaving variables residual and untouched, as its model in the main paper's Definition "Constant Removal" requires ("the instance layer is unchanged").*

So the mode scoping of Finding 9 costs the main paper nothing.

4.2 Named instantiations (`TSInst`)

$\mathsf{instantiate}(n, \bar{a})$ dispatches on the definition of n (`TEnv::instantiate`): a `TypeDef` binds each type parameter P_i to the argument a_i *as a type term* in the lexical `bindings` map (checking constraints on a throwaway clone, diagnostics only) and normalizes the alias body — a reference to P_i in the body resolves to the binding by `boundTy.withPos(pos)`, which changes only the position and not the hash, so the argument term is inserted *verbatim*; a `Class` builds the object type via `new`/`newObject`/`addFields` with the same binding mechanism for field types; two early returns re-emit the unexpanded TSInst itself — the synthetic missing-annotation placeholder, and the recursion guard when n is already being expanded (`seenInstantiate`).

impl: `TEnv.sk / instantiate (110--148), instantiateTParams (174), normalize TAInst arm (1354--1428), new (325), newObject (359)`

What normalization computes from the body is `TEnv` territory: this paper does not model `normalize` and instead names the interface assumption that the containment needs.

Axiom 11 (E-NORM: Instantiation is Semantics-Preserving)**.** *For every named definition n and arguments $\bar{a}$, and every state ρ at which the expansion runs,* $[\![\mathsf{TSInst}(n, \bar{a})]\!]_\rho \subseteq [\![\mathsf{instantiate}(n, \bar{a})]\!]_\rho$. *For the two early returns the result* is *the input, so the axiom is trivially true there and the re-emitted instance is a residual that keeps its semantics by construction.*

impl: `TEnv.sk / instantiate, normalize`

Under **E-NORM**, the `TSInst` one-step containment reduces to one further claim: that reusing a *cached* expansion — computed at some earlier state ρ_0 — denotes at ρ' no less than a fresh instantiate run at ρ' would. That is the centerpiece (Section 5); Theorem 18 discharges it for the alias path by showing the cached and fresh values are *equal*.

4.3 Unexpanded intersections (`TInter`)

The one-step expansion of $\mathsf{TInter}(\{t_1, \ldots, t_k\})$ sets `tenv.expandInter` and left-folds `TType::inter` over the members. (That flag selects full over deferred field intersection; its path-scoping and the deferred form's containment are registry row TENV-EXPANDINTER (OPEN), which this arm's argument imports.) By (I1) and **L-INTER** (paper B, which covers `inter` at every level, including the `seenInter` cache and the cycle guard `tenv.seenInter`),

$$[\![\mathsf{TInter}(\{t_i\})]\!]_\rho = \bigcap_i [\![t_i]\!]_\rho \subseteq [\![\mathsf{interN}(\{t_i\})]\!]_{\rho'},$$

with the fold's intermediate evolutions transported by **L-MONO**. When `inter_` hits an in-progress pair it returns the *unexpanded* intersection `unexpandedInter` (`TTypeInter.sk:154--156`); by (I1) that residual keeps its semantics exactly. Similarly the bare-variable arms return a variable occurrence with an added constraint — a residual whose semantics (V1)-intersects the operand, again exact for our direction.

impl: `TTypeInter.sk / TInter::expand (37), inter (97), inter_ (129), unexpandedInter (82)`

4.4 Delayed array literals (`TArrayLit`)

`ArrayLitTyping::expand` returns the committed type $\mathsf{peek}(\alpha)$ — the binding of the literal's commitment variable intersected with its constraints, read *without* any state transition — when α is bound, and otherwise the mutable tuple type $\mathsf{Tuple}[elts]$ built from the widened element types. The containment needs the commitment discipline documented at the top of `TArrayLitTyping.sk` ("every commitment uses the target type or the widened element types"), which is array-literal-typing territory; we name it:

Axiom 12 (E-ARR: Array-Literal Commitment Discipline)**.** *For every* $\mathsf{TArrayLit}(\alpha, elts, _)$ *and state ρ: (i) if ρ binds α then* $[\![\mathsf{TArrayLit}(\ldots)]\!]_\rho \subseteq [\![\mathsf{peek}(\alpha)]\!]_\rho$ *(the committed type contains the literal's values); (ii) if α is unbound then* $[\![\mathsf{TArrayLit}(\ldots)]\!]_\rho \subseteq [\![\mathsf{Tuple}[elts]]\!]_\rho$ *(the literal's element values inhabit the widened element types — a well-typedness fact about how `TArrayLit` is constructed). Clause (ii) must hold at every state, including states where α later becomes bound: the tuple view taken by an early expansion must not be invalidated by a later, wider commitment (e.g. to `Array<U>`).*

impl: `TArrayLitTyping.sk / expand (100), arrayLitCommittedType (85)`

impl: `TArrayLitTyping.sk / arrayLitToTupleType (113); TTypeSubst.sk / peek (224)`

4.5 The containment theorem

Theorem 13 (T-EXP)**.** *For $m \in \{\mathsf{none}, \mathsf{cov}\}$, all t, ρ: with $(I', C', \rho') = \mathsf{expand}_m(t, \rho)$,*

$$[\![t]\!]_\rho \subseteq [\![(I', C')]\!]_{\rho'},$$

residual instances appear verbatim in I' and keep their semantics, and the state evolution $\rho \to \rho'$ consists only of the transitions of Lemma 5. The result is conditional on: ***L-UNION/L-INTER*** *(paper B),* ***L-MONO*** *(main paper),* ***E-NORM***, ***E-ARR***, *and — for reuse of cached* `TSInst` *expansions — Theorem 18 on the alias path and Obligation 24 on the class path.*

Proof. Lemma 6 reduces the claim to the one-step containment ($*$), established per construct: Lemma 8 (variables), Axiom 11 plus Theorem 18 (named instantiations, cached or fresh), Section 4.3 (intersections), Axiom 12 (array literals). □

5 Centerpiece: Soundness of the `seenExpansion` Cache

The cache maps a TSInst to its expanded body and is consulted before every instantiate call (Definition 4). The code comment (`TTypeSubst.sk:17--31`) argues reuse stays sound as inference proceeds because the cached body substitutes each type parameter *symbolically*: arguments appear as free variables, re-resolved at every use through `Subst::get`; union/inter never resolve an unbound `TVar`; and the one construct that reads a binding during expansion, `keyof`, approximates identically whether or not the variable is bound. This section makes those claims precise, proves a strictly stronger statement for the alias path, and corrects the comment in two places.

5.1 Symbolic arguments, precisely

Definition 14 (Symbolic in the Arguments)**.** $\mathsf{instantiate}(n, \bar{a})$ *is* symbolic in the arguments *if there is a type-with-holes $B_n[\cdot_1, \ldots, \cdot_k]$, determined by the definition of n alone, such that for every $\bar{a}$ and every state ρ, $\mathsf{instantiate}(n, \bar{a})$ run at ρ returns $B_n[\bar{a}]$ — the same* TType value *— where each hole occurrence is filled by the argument term a_i either (i) verbatim (up to position, which is not hashed), or (ii) as a variable occurrence of a_i's variable with occurrence-level constraints added (`setConstraint` — a functional update, registry row SETCONSTRAINT-FUN), or (iii) replaced by a* binding-independent *approximation (a fixed type not depending on ρ). Diagnostics emitted along the way may depend on ρ; the returned value may not.*

Lemma 15 (Symbolic ⇒ Cache-Reuse Exactness)**.** *If $\mathsf{instantiate}(n, \bar{a})$ is symbolic in the arguments, then for any states ρ_0 (cache fill) and ρ' (use), the cached value equals what a fresh run at ρ' would return; hence $[\![\mathit{cached}]\!]_{\rho'} = [\![\mathit{fresh}]\!]_{\rho'}$ and the* `TSInst` *one-step containment under reuse follows from* ***E-NORM*** *at ρ' alone.*

Proof. Immediate: both runs return the value $B_n[\bar{a}]$. The point of the definition is that argument *bindings* are never consulted, so the variables inside $\bar{a}$ remain free in the cached term and are re-resolved through get at every later expansion step of the *user* of the cached term — never frozen into the cache. □

5.2 The alias path is binding-blind

The mechanism turns out to be simpler and stronger than the comment's operation-by-operation argument: on the `TypeDef` path, normalization *never consults the live substitution at all.*

Definition 16 (Binding-Blind)**.** *A computation is* binding-blind *if every* `Subst` *whose binding map it consults (via* `get`*,* `peek`*,* `isUnbound`*,* `isOpen`*,* `isClosed`*) is freshly created with the empty state, or is a clone whose results influence diagnostics only.*

Lemma 17 (normalize is Binding-Blind)**.** `TEnv::normalize` *and the helpers it reaches on the* `TypeDef` *path are binding-blind. By arms of the dispatch (*`TEnv.sk:1104--1445`*):*

- Literals, primitives, top, bottom, tuples-as-names, and the fatal arms*: these construct constants or* `TType::inst`*; no substitution is involved.*
- `TAInst` *(1354): resolves through the lexical* `bindings` *map — a* `TEnv` *field, not a substitution — returning the bound term via* `withPos` *(verbatim insertion, case (i) of Definition 14); or emits an unexpanded* `TType::inst`*; the reference-site constraint check (1419–1426) runs on* `Subst::mcreate(tcenvb)` — empty state — *and affects diagnostics only.*
- `TAUnion`/`TAInter` *(1429–1444): both create a fresh empty* `Subst` *(1430, 1438).* `union` *never reads bindings at any level (Lemma 21).* `inter` *under the empty state sees every variable as unbound: bare-variable operands take the* `setConstraint`/`unexpandedInter` *arms (case (ii)); variables nested in compound operands are sealed to top* in the throwaway state *and thus replaced by the binding-independent* $\top$ *(case (iii)) — see Finding 22.*
- `TAKeyOf` *(1172–1175): creates a fresh empty* `Subst` *(1173) and expands the operand under it; see Finding 23.*
- `TAObject`/`TAFun`/`TATypeOf`/tuples*: recursive normalization plus fresh empty substitutions at every operation site (*`TEnv.sk:1451, 1556, 1567, 1582, 1639, 1752, 1893`*); the mapped-type key analysis (*`FNIn`*, 1953) expands the key type in mode* none *under a fresh empty* `Subst` *(1893, 1973), leaving variables residual and unread.*

impl: `TEnv.sk / normalize (1104), Subst creation sites (1173, 1421, 1430, 1438, 1451, 1556, 1567, 1582, 1639, 1752, 1893, 1973)`

Theorem 18 (Alias-Path Expansion Invariance)**.** *For a* `TypeDef` n*,* instantiate$(n, \bar{a})$ *is symbolic in the arguments (Definition 14); in fact it returns the same* TType *value at every state. Consequently (Lemma 15)* `seenExpansion` *reuse is* exact *on the alias path: the cached body denotes, at every later state, exactly what a fresh recomputation would.*

Proof. instantiate on the `TypeDef` arm performs: parameter binding (`instantiateTParams`) — the returned bindings are the argument terms themselves, defaults normalized recursively, and the constraint check runs on `subst.clone()` of the state *copy* and is discarded (`TEnv.sk:222`, diagnostics only); then `normalize` of the body, which is binding-blind by Lemma 17. A binding-blind computation's result cannot depend on the ambient binding map; the remaining inputs (definition ASTs via `defMap`, the lexical bindings, the argument terms) are fixed by $(n, \bar{a})$ and the compilation unit. The memo caches consulted along the way (`seenInter`/`seenUnion`) belong to the *fresh empty* substitutions of Lemma 17 and start empty at each site, so they cannot import state-dependence. □

Corollary 19 (Immunity to the Freeze-Window Hazard)**.** *Papers B/E of this series identify a hazard family in the* `Subst` *memo caches: under* `freezeCandidates`*,* `Subst::get` *returns an open candidate's bound* without sealing it*, so a cache entry whose value depends on that read (*`seenSub`*,* `seenInter`*) can go stale when the candidate later grows. On the alias path,* `seenExpansion` *is* immune by construction*: by Theorem 18 the cached value does not depend on any binding read, frozen or not — the unsealed read can return whatever it likes without being able to reach the cached value. (The class path is exactly where this immunity argument breaks; Obligation 24.)*

Corollary 20 (Coherence Under save/restore/clone)**.** *`seenExpansion` is part of `SubstState`: `save`, `restore`, and `clone` carry bindings and cache atomically, so an entry filled on an exploratory path is rolled back with that path's bindings, and entries filled in a clone die with the clone. On the alias path this atomicity is belt-and-braces — by Theorem 18 even a leaked entry would be correct — but it is what keeps the* class-path *exposure bounded to reuse within one live substitution's lifetime.*
impl: `TTypeSubst.sk / SubstState::restore (40), SavedSubst::create (63), Subst::clone (137)`

5.3 Verifying the comment's three claims

Lemma 21 (Union Never Resolves a TVar)**.** *`TType::union` and everything it reaches perform no binding reads: `union_` unions the instance sets verbatim (`TInsts::union`) and merges clauses by var-set key; the recursive field/function cases (`TField::union`, `TFun::union`) recurse through `union` on covariant components — and through `inter` on contravariant ones, which is where the next lemma takes over. The `seenUnion` cache comment ("any TVar in the operands passes through unchanged") is confirmed at the `TType` level.*
impl: `TTypeUnion.sk / union (22), union_ (42), TInsts::union (167)`

Finding 22 (The comment's `inter` claim is imprecise)**.** *We attempted to verify "inter never resolves an unbound TVar" verbatim and* failed*: it is true only for* bare *variable operands (`inter_`'s guard arms, `TTypeInter.sk:138--153`, which attach the other operand as an occurrence constraint). A variable nested in a* compound *operand — e.g. (α | `undefined`) & `string` — falls through to `expandToClauses(Covariant())` on both operands (`TTypeInter.sk:158--159`), which* does *resolve α: sealing it to top if unbound, reading its value if bound. The claim is rescued where the cache needs it by Lemma 17: during alias normalization `inter` runs under a fresh* empty *state, so the nested variable is always unbound there, the seal lands in the throwaway state, and the substituted $\top$ is binding-independent — and replacing a conjunct by $\top$ over-approximates the intersection, so **E-NORM**'s direction is unharmed. Outside normalization (live `inter` during checking) the compound-operand resolution is real but is paper B's subject, not the cache's. Suggested action: reword the comment's parenthetical to "union caches independently of growth; inter attaches* bare *operand variables as constraints, and during normalization runs on an empty substitution, so nested variables are top-approximated, never bound-resolved."*

Finding 23 (The `keyof` claim holds vacuously — it never reads)**.** *The comment says `keyof` "approximates an abstract or unresolved operand identically (`string | number`) whether or not the variable is bound." The audit shows something stronger: the `TAKeyOf` arm creates a fresh* empty *`Subst` (`TEnv.sk:1173`) and `keyOfType` expands the operand under it, so a variable argument is* always *treated as unbound — its covariant read seals it to top in the throwaway state, the expansion has a top clause, and the guard at `TEnv.sk:2197--2206` returns `string | number`. The live binding is unreachable, so cached and fresh values agree not because the two reads approximate identically but because there is no read. (The result also covers rigid parameters, whose clauses have nonempty var-sets — the same guard.) Two caveats, both outside the cache theorem: `keyOfType`'s own over-approximation contract explicitly excludes validity checking ("do not use keyof to check that a key is valid," `TEnv.sk:2188--2193`), and symbol keys of an unresolved operand are not covered by `string | number` — both are **E-NORM**-side concerns about what the approximation means, not about cache invariance.*
impl: `TEnv.sk / normalize TAKeyOf arm (1172--1175), keyOfType (2195)`

5.4 Where the proof stops: the class path

Obligation 24 (E-CLASS-SYM: Class Instantiation Symbolicity)**.** *On the* `Class` *arm,* instantiate *calls* `new` → `newObject` → `addFields` *passing the* copy of the live state *made at* `TEnv.sk:135` *— not a fresh empty substitution. Field-type normalization inside is binding-blind (Lemma 17 applies arm by arm), but member assembly is not obviously so: any* `TType::inter` *reached with that substitution — the audited candidates are index merging (*`TField::interIndex`*,* `TTypeInter.sk:245`*, called unconditionally on* `TType::inter`*, unlike* `TField::inter`*, which defers via* `unexpandedInter` *when* `tenv.expandInter` *is false), constructor assembly (*`findConstructor`*), and* `TIndexes::create` *— can reach* `expandToClauses(Covariant())` *on a compound operand mentioning a type argument's variable and read its* current *binding. Because the read happens in the discarded copy, the seal that would normally freeze the variable and justify the read (paper's never-widen lemma (iii)) is thrown away: the live variable can still grow, and the cached class body would then diverge from a fresh recomputation — the same staleness family as the freeze-window hazard (Corollary 19), reachable here even without* `freezeCandidates`*, and additionally through it.* ***Status: open.*** *We attempted both directions and completed neither: the proof attempt dies at the* `interIndex` *chain above, and a concrete end-to-end counterexample (a class instantiated at a bound-open candidate whose member merge reads the bound, cached, followed by observable growth) was not constructed. Discharge either by (a) showing the reachable* `inter` *operands during* `addFields` *never nest a live inference variable inside a compound type (targ variables occur bare or inside unexpanded* `TSInst`*s there), or (b) making the class path binding-blind by construction (fresh empty substitution in* `newObject`*, as the alias path already effectively has), or (c) keying* `seenExpansion` *entries for class instances by substitution version, as* `seenSub` *already is for cycles.*

impl: `TEnv.sk / instantiate (135, 141), newObject (359), addFields (559)`

impl: `TTypeInter.sk / TField::interIndex (245), inter_ (158)`

Obligation 25 (E-GUARD: Recursion-Guard Identities Must Not Poison the Live Cache)**.** instantiate *returns the unexpanded* TSInst *itself when the guard set* `seenInstantiate` *already contains n (*`TEnv.sk:128--130`*), and* `expandToVectors` *caches whatever* instantiate *returns (*`TType.sk:653`*). If such an* identity *entry were ever written to a long-lived substitution's cache, every later expansion of that instance would hit the identity, whose sole instance is then dropped by the seen check — the named type would contribute* nothing *(effectively ⊥) to every future expansion, violating T-EXP outright. For the* recursion-guard *identity return this is empirically safe:* `seenInstantiate` *is nonempty only inside an* instantiate *frame using the discarded substitutions of Lemma 17, and a recursive alias's inner* `TSInst` *sits nested in a field/argument the head worklist never processes (*`type A = A | number` *reads as* `number`*).* ***Status of the recursion-guard case: open (small).*** *But there is a* second *identity return — the missing-annotation placeholder (*`TEnv.sk:122--124`*) — that fires with* `seenInstantiate` empty*, at a top-level live-cache-fill site; the re-emitted identity equals the popped instance and is dropped by the expansion seen-set, leaving empty clauses, so an un-annotated ambient-class field expands to ⊥ and reads into any type including* `never`*. That is registry row* `E-GUARD-PLACEHOLDER`*,* VIOLATED *and witnessed end-to-end (issue draft* `ambient-unannotated-field-bottom.md`*); the fix keeps an identity* `instantiate` *result as a residual (or returns a top clause) rather than letting the seen-set drop it. A cache-write invariant that only asserts* `seenInstantiate` *is empty does* not *catch the placeholder (it is empty there); the invariant must instead assert "the cached value's insts ≠ the key."*

impl: `TEnv.sk / instantiate (125--130); TType.sk / expandToVectors (646--656)`

6 Alias Cycles and Termination

Three guards keep expansion of recursive definitions finite; soundness of each cutoff is the point here, termination secondary.

One-level unfolding (`seenInstantiate`). While expanding n, an inner reference to n normalizes to an unexpanded `TSInst` instead of unfolding (`TEnv.sk:45--60, 128--131, 1405--1407`). The cutoff is sound by construction: the re-emitted instance denotes the named type ((N1)), and nested occurrences sit inside object fields or targs, where the worklist does not descend — expansion is one level deep by design (the main paper's "expansion rewrites a type to its instance-plus-clauses view" is inherently head expansion).

The worklist *seen* set. A *top-level* recursive occurrence — `type A = A | number` normalizes to $(\{\mathsf{TSInst}(A)\}, \{\texttt{number}\})$ — is deduplicated: the body's $\mathsf{TSInst}(A)$ equals the instance being expanded and is dropped (`TType.sk:671--675`). The expansion of A is then just `number`. This is sound under a least-fixpoint reading of type definitions: the equation $X = X \cup [\![\texttt{number}]\!]$ has least solution $[\![\texttt{number}]\!]$, and any value's membership in a recursively defined type must be justified by finite unfolding. We record this reading as part of (N1)'s meaning rather than as a separate assumption; a definition intended coinductively (values justified only by infinite regress) has no inhabitants a program can construct. Mutual cycles behave the same: expanding `type A = B; type B = A` yields empty clauses (bottom) after both identities deduplicate — the least fixpoint — while the checker independently rejects such definitions at validation (TS2456).

Fatal circularity (`forbiddenInsts`). Within one instantiate frame, a *second* reference to the in-expansion name is fatal (TS2456, `TEnv.sk:1375--1379`): the first reference marks the name forbidden (1405–1407), the next lookup dies. Generative loops (`type A<T> = A<A<T>> | number`) die the same way — normalizing the inner argument `A<T>` is a second reference. A fatal diagnostic ends checking, which is trivially sound. Termination of the worklist itself: each iteration either pops a *seen* instance's expansion products (each distinct instance is processed at most once) or returns $\top$; non-termination would require instantiate to keep manufacturing distinct instances at the top level of bodies, which the generative-loop fatal cuts off on the alias path. We have not proved a global termination bound for the class path (chains of distinct `TSInst`s through superclass clauses); termination is outside the soundness contract and is left unstated.

impl: `TEnv.sk / seenInstantiate (60), forbiddenInsts (64), normalize TAInst (1375, 1405); TType.sk / expandToVectors (671--675)`

7 Status and Ledger Edits

7.1 Decomposition of L-EXP

Item	Statement	Anchor
E-PLUMB	Worklist, clause rebuild, `mapSTypes` factoring, top short-circuit, residual pass-through (Lemmas 6, 7)	`expandToVectors`, `mapSTypes`

Item	Statement	Anchor
E-TVAR	Variable one-step containment, modes none/cov (Lemma 8); mode contra excluded with counterexample (Finding 9); refinement clients verified none/cov-only (Lemma 10)	`Subst::get`
E-INTER	`TInter` one-step containment; cycle residual exact (§4.3)	`TInter::expand`
E-NORM	Instantiation is semantics-preserving at the state of use (Axiom 11) — the `TEnv`/normalize interface	`TEnv::instantiate`
E-ARR	Array-literal commitment discipline (Axiom 12)	`TArrayLitTyping.sk`
E-SYM-ALIAS	Cache invariance, `TypeDef` path: binding-blind, hence exact reuse and freeze-window immunity (Theorem 18, Corollary 19)	`seenExpansion`; `TEnv::normalize`
E-CLASS-SYM	Cache invariance, `Class` path (Obligation 24)	`TEnv::newObject`; `TField::interIndex`
E-GUARD	Recursion-guard identities never cached into a long-lived substitution (Obligation 25)	`expandToVectors` (653)

Net effect on **L-EXP**: ASSUMED → PROVED for modes none/cov, conditional on **L-UNION**/**L-INTER**, **L-MONO**, **E-NORM**, **E-ARR**, **E-CLASS-SYM**, **E-GUARD**. The unconditional residue is smaller and sharper than the original axiom: two named interface assumptions and two localized open obligations.

7.2 Registry

The authoritative, current statuses are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence — in particular for the mode scoping of Axiom **L-EXP** (the containment holds for modes `None`/`Covariant` only, Lemma 10) and for the rows E-NORM, E-ARR, E-CLASS-SYM, E-GUARD. The registry rows this paper supports:

Row	Statement	Status
L-EXP	Expansion preserves or enlarges semantics (modes `None`/Covariant); residual instances keep theirs.	PROVED
E-NORM	Named-type normalization is semantics-preserving (interface consumed by expansion).	ASSUMED
E-ARR	Delayed array-literal forms keep their semantics under expansion.	ASSUMED
E-CLASS-SYM	The class path of expansion is binding-blind (or its binding reads are harmless).	OPEN
E-GUARD	Recursion-guard identity expansions are never cached into long-lived substitutions.	OPEN
E-GUARD-PLACEHOLDER	An un-annotated ambient-class field placeholder must not head-expand to $\perp$ (the case E-GUARD's recursion-guard audit did not cover).	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-UNION	`union` over-approximates set union (all levels).	B	PROVED
L-INTER	`inter` over-approximates set intersection (the containment clause).	B	PROVED
L-MONO	Refinement-time state evolutions preserve membership in already-installed types.	E	PROVED
TVAR-SEM	Sharpened variable denotation (get-time constraint intersection as definition), verified against every consumer.	B	OPEN
SETCONSTRAINT-FUN	`TVar::setConstraint` is a functional update on a frozen value: constraints are occurrence-level, not shared state.	E	PROVED
TENV-EXPANDINTER	The `tenv.expandInter` mode flag is path-scoped, and both intersection forms it selects (expanded and deferred `unexpandedInter`) satisfy the L-INTER containment.	E	OPEN

Outstanding recommendations. (i) Fix the code comment `TTypeSubst.sk:17--31` per Finding 22 (inter's "never resolve" holds for bare operands; nested variables are top-approximated because normalization substitutions are empty) and Finding 23 (keyof never reads the live binding — `TEnv.sk:1106`). (ii) Wire the E-GUARD pinned test; record Lemma 5's inventory (expansion performs get-transitions and cache writes only; never a grow) as the concrete content behind L-MONO's "evolutions of the kind covered".

7.3 Findings from failed proof attempts (index)

Finding 9: unconditional (all-modes) L-EXP is false; counterexample unbound-variable contravariant read returning $\bot$. Finding 22: the comment's blanket "inter never resolves an unbound TVar" failed verification; rescued only via the binding-blind property. Finding 23: the keyof-equal-approximation claim could not be proved as stated (a bound variable read *would* produce precise keys) — it is true for the vacuous reason that no live read occurs. Obligation 24: classpath symbolicity resisted both proof (dies at the `interIndex` chain) and refutation (no end-to-end counterexample constructed). Obligation 25: cache poisoning by recursion-guard identities could not be excluded by a local invariant; global call-site audit only.

Implementation Anchors (Summary)

Model	**Implementation (rev. `cf5a6bc7`)**
expand_m, worklist	`TType.sk / expandToVectors (634)`
rebuilds	`expand (680), expandToClauses (692), mapSTypes (388), rawMap (362)`
get transitions, freeze	`TTypeSubst.sk / Subst::get (188), freezeCandidates (89)`
cache + saved state	`TTypeSubst.sk / seenExpansion (31), restore (40), clone (137)`
cache read/write site	`TType.sk / expandToVectors (646--656)`
instantiate, parameter binding	`TEnv.sk / instantiate (110), instantiateTParams (174)`
symbolic resolution	`TEnv.sk / normalize TAInst arm (1354--1373), bindings (43)`
fresh empty substitutions	`TEnv.sk / 1173, 1421, 1430, 1438, 1451, 1556, 1567, 1582, 1639, 1752, 1893, 1973`
keyof	`TEnv.sk / keyOfType (2195)`
mapped types	`TEnv.sk / FNIn arm (1953)`
class path	`TEnv.sk / new (325), newObject (359), addFields (559)`
intersections	`TTypeInter.sk / TInter (17), expand (37), inter_ (129), interIndex (245)`
unions	`TTypeUnion.sk / union (22), union_ (42)`
array literals	`TArrayLitTyping.sk / expand (100)`
recursion guards	`TEnv.sk / seenInstantiate (60), forbiddenInsts (64)`